Django Class-Based Views (CBV) — Complete Notes

Topics: Django Views, Generic Views, CRUD, Mixins, Authentication, Authorization, Method Overriding, Request Lifecycle, Query Optimization

Level: Beginner to Advanced

Language: English

---

1. Introduction to Class-Based Views

1.1 What Is a Class-Based View?

A **Class-Based View (CBV)** is a Django view implemented using a Python class instead of a function.

A view receives an HTTP request and returns an HTTP response.

Django supports two main approaches to implementing views:

1. Function-Based Views (FBVs)
2. Class-Based Views (CBVs)

Function-Based View (FBV)

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Hello, Django!")
```

Class-Based View (CBV)

```
from django.http import HttpResponse
from django.views import View


class HomeView(View):
    def get(self, request, *args, **kwargs):
        return HttpResponse("Hello, Django!")
```

In this example, `HomeView` is a class, and its `get()` method handles GET requests.

1.2 Advantages of CBVs

1. Code Reusability

Existing view functionality can be reused through inheritance and mixins.

2. Inheritance

You can extend an existing view and override its attributes or methods.

3. Generic Views

Django provides built-in views for common tasks, reducing repetitive code.

4. Method-Based Request Handling

Different HTTP methods can be handled through separate methods, such as `get()` and `post()`.

5. Maintainability

CBVs help organize larger applications and separate responsibilities.

6. CRUD Support

Generic views simplify creating, reading, updating, and deleting database objects.

1.3 FBV vs. CBV

Feature	Function-Based View	Class-Based View
Structure	Python function	Python class
Request handling	Conditional statements	HTTP handler methods
Code reuse	Functions and helpers	Inheritance and mixins
Generic views	Common behavior often	Built-in generic views

	requires manual implementation	
Simple endpoints	Often straightforward	May require additional structure
Complex CRUD	May require more code	Often requires less code
Learning curve	Generally easier initially	Requires understanding inheritance and method resolution

Important: Neither approach is always better. Choose the approach that best fits your application.

---

2. Basic Class-Based Views

Django provides the base `View` class in:

```
from django.views import View
```

2.1 Creating a Basic CBV

```
from django.http import HttpResponse
from django.views import View
```

```
    class MyView(View):

        def get(self, request, *args, **kwargs):
            return HttpResponse("This is a GET request.")

        def post(self, request, *args, **kwargs):
            return HttpResponse("This is a POST request.")
```

Common HTTP Handler Methods

Method	Purpose
`get()`	Handles GET requests
`post()`	Handles POST requests
`put()`	Handles PUT requests when implemented
`patch()`	Handles PATCH requests when implemented
`delete()`	Handles DELETE

	requests when implemented
`head()`	Handles HEAD requests
`options()`	Handles OPTIONS requests
`trace()`	Not provided as a standard handler by Django's base `View`

These methods must be implemented when the application needs the corresponding behavior.

Django does not automatically implement application-specific PUT, PATCH, or DELETE operations.

2.2 URL Configuration

```
from django.urls import path
from .views import MyView


urlpatterns = [
    path("myview/", MyView.as_view(), name="myview"),
]
```

Notice the use of:

```
MyView.as_view()
```

You must call `as_view()` when connecting an ordinary class-based view to a URL pattern.

---

3. Understanding `as_view()`

The `as_view()` method converts a CBV into a callable that Django's URL resolver can invoke.

Example:

```
path("home/", HomeView.as_view(), name="home")
```

3.1 CBV Request Lifecycle

```
Browser Request
       |
       v
URL Resolver
       |
       v
HomeView.as_view()
       |
       v
Create View Instance
       |
       v
setup()
       |
       v
dispatch()
       |
       v
get() / post() / other supported handler
       |
       v
HTTP Response
       |
```

```
            v
      Browser
```

Important Methods

`as_view()`

Creates a callable view from the class.

`setup()`

Initializes the view instance with the request and URL arguments.

`dispatch()`

Selects the appropriate handler based on the HTTP request method.

For example:

- GET request → `get()`
- POST request → `post()`
- PUT request → `put()`, if implemented

If the request method is unsupported, Django normally returns HTTP 405 Method Not Allowed.

`http_method_not_allowed()`

Handles unsupported HTTP methods.

`options()`

Provides the default OPTIONS response.

---

4. Class Attributes and Method Overriding

CBVs can be configured by overriding class attributes and methods.

4.1 Overriding Class Attributes

```
from django.http import HttpResponse
from django.views import View
```

```
class GreetingView(View):
    greeting = "Good Day"

    def get(self, request, *args, **kwargs):
        return HttpResponse(self.greeting)


class MorningGreetingView(GreetingView):
    greeting = "Good Morning"
```

Here, `MorningGreetingView` inherits from `GreetingView` and overrides the `greeting` attribute.

4.2 Passing Attributes Through `as_view()`

You can also configure class attributes in the URL configuration.

```
from django.urls import path
from .views import GreetingView


urlpatterns = [
    path(
        "greeting/",
        GreetingView.as_view(greeting="Hello, Django!"),
        name="greeting",
    ),
]
```

The value passed to `as_view()` overrides the class attribute for that view configuration.

Important: These configuration arguments must correspond to existing attributes on the view class. They are configured when `as_view()` is called, not separately for each request.

---

5. Built-in Generic Class-Based Views

Django provides generic CBVs for common web development tasks.

The main generic views are:

View	Purpose
`View`	Basic request handling
`TemplateView`	Renders a template
`RedirectView`	Redirects to another URL
`FormView`	Handles forms
`ListView`	Displays multiple objects
`DetailView`	Displays one object
`CreateView`	Creates an object
`UpdateView`	Updates an object
`DeleteView`	Deletes an object

Generic views reduce repetitive code and make applications easier to maintain.

5.1 TemplateView

`TemplateView` renders a template.

views.py

```
from django.views.generic import TemplateView


class HomeView(TemplateView):
    template_name = "home.html"
```

urls.py

```
from django.urls import path
from .views import HomeView


urlpatterns = [
    path("", HomeView.as_view(), name="home"),
]
```

templates/home.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>
    <h1>Welcome to Django</h1>
</body>
</html>
```

Adding Context Data

You can override `get_context_data()` to provide additional information to the template.

```
from django.views.generic import TemplateView


class HomeView(TemplateView):
    template_name = "home.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
```

```
            context["message"] = "Welcome to Django CBV"
            return context
```

Template:

```
<h1>{{ message }}</h1>
```

Use cases: Home pages, About pages, landing pages, and other template-oriented pages.

5.2 RedirectView

`RedirectView` redirects users to another URL.

Using a URL

```
from django.views.generic import RedirectView


class HomeRedirectView(RedirectView):
    url = "/home/"
```

Using a Named URL Pattern

```
from django.views.generic import RedirectView


class HomeRedirectView(RedirectView):
    pattern_name = "home"
```

URL configuration:

```
path(
    "go-home/",
    HomeRedirectView.as_view(),
    name="go-home",
)
```

- `url` specifies the destination URL directly.
- `pattern_name` specifies the name of a URL pattern.

Prefer named URLs when practical.

---

6. Generic Views for Database Objects

Django provides generic views for displaying database records and performing CRUD operations.

The examples below assume that a `Product` model exists.

6.1 Example Product Model

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    description = models.TextField(blank=True)
    active = models.BooleanField(default=True)

    def __str__(self):
        return self.name
```

Create and apply migrations after defining the model.

```
python manage.py makemigrations
python manage.py migrate
```

6.2 ListView

`ListView` retrieves multiple objects and displays them in a template.

views.py

```
from django.views.generic import ListView
from .models import Product


class ProductListView(ListView):
    model = Product
    template_name = "products/product_list.html"
    context_object_name = "products"
```

urls.py

```
from django.urls import path
from .views import ProductListView
```

```
urlpatterns = [
    path(
        "products/",
        ProductListView.as_view(),
        name="product-list",
    ),
]
```

Template

`templates/products/product_list.html`

```
<h1>Product List</h1>

<ul>
    {% for product in products %}
        <li>
            {{ product.name }} - {{ product.price }}
        </li>
    {% empty %}
        <li>No products found.</li>
    {% endfor %}
</ul>
```

Important Attributes

Attribute	Purpose
`model`	Specifies the model
`queryset`	Specifies the objects to retrieve
`template_name`	Specifies the template

`context_object_name`	Sets the context variable name
`paginate_by`	Sets the number of objects per page

By default, the template receives `object_list`. Django may also provide a model-specific variable, such as `product_list`.

6.3 DetailView

`DetailView` retrieves and displays one object.

views.py

```
from django.views.generic import DetailView
from .models import Product


class ProductDetailView(DetailView):
    model = Product
    template_name = "products/product_detail.html"
    context_object_name = "product"
```

urls.py

```
path(
    "products/<int:pk>/",
    ProductDetailView.as_view(),
    name="product-detail",
)
```

Template

`templates/products/product_detail.html`

```
<h1>{{ product.name }}</h1>

<p>Price: {{ product.price }}</p>

<p>{{ product.description }}</p>
```

Django uses the URL's `pk` or a configured slug to retrieve the object.

If the object does not exist, Django normally returns a 404 response.

---

7. CRUD Operations with Generic Views

CRUD stands for:

- **Create:** Create a new object.
- **Read:** Retrieve and display objects.
- **Update:** Modify an existing object.
- **Delete:** Remove an existing object.

Django provides generic views that simplify these operations.

7.1 CreateView

`CreateView` displays a form and creates a model instance after successful validation.

views.py

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Product



class ProductCreateView(CreateView):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")
```

urls.py

```
path(
    "products/create/",
    ProductCreateView.as_view(),
```

```
        name="product-create",
    )
```

Template

`templates/products/product_form.html`

```
<h1>Product Form</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Save</button>
</form>
```

CreateView Workflow

```
GET Request
     |
     v
Display Empty Form
     |
     v
User Submits Form
     |
     v
POST Request
     |
     v
Form Validation
     |
     +---- Invalid ----> Display Form Errors
     |
     +---- Valid ------> Save Object
                              |
                              v
                          Redirect
```

`CreateView` automatically constructs a suitable `ModelForm` when configured correctly.

7.2 UpdateView

`UpdateView` retrieves an existing object and updates it after successful form validation.

```
from django.urls import reverse_lazy
from django.views.generic import UpdateView
from .models import Product


class ProductUpdateView(UpdateView):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")
```

URL configuration:

```
path(
    "products/<int:pk>/update/",
    ProductUpdateView.as_view(),
    name="product-update",
)
```

`UpdateView` uses the URL parameter to identify the existing object.

7.3 DeleteView

`DeleteView` displays a confirmation page and deletes the object when the confirmation form is submitted.

```
from django.urls import reverse_lazy
from django.views.generic import DeleteView
from .models import Product


class ProductDeleteView(DeleteView):
    model = Product
    template_name = "products/product_confirm_delete.html"
    success_url = reverse_lazy("product-list")
```

URL configuration:

```
path(
    "products/<int:pk>/delete/",
```

```
        ProductDeleteView.as_view(),
        name="product-delete",
    )
```

Template

`templates/products/product_confirm_delete.html`

```
<h1>Delete Product</h1>

<p>
    Are you sure you want to delete {{ object.name }}?
</p>

<form method="post">
    {% csrf_token %}

    <button type="submit">Confirm Delete</button>
</form>
```

Important: `DeleteView` normally displays the confirmation page on GET and performs deletion on POST. Never perform destructive operations through GET requests.

7.4 Complete CRUD URL Configuration

```
from django.urls import path

from .views import (
    ProductListView,
    ProductDetailView,
    ProductCreateView,
    ProductUpdateView,
    ProductDeleteView,
)



urlpatterns = [
    path(
        "products/",
        ProductListView.as_view(),
        name="product-list",
    ),
    path(
```

```
            "products/create/",
            ProductCreateView.as_view(),
            name="product-create",
        ),
        path(
            "products/<int:pk>/",
            ProductDetailView.as_view(),
            name="product-detail",
        ),
        path(
            "products/<int:pk>/update/",
            ProductUpdateView.as_view(),
            name="product-update",
        ),
        path(
            "products/<int:pk>/delete/",
            ProductDeleteView.as_view(),
            name="product-delete",
        ),
    ]
```

The corresponding template files must exist, and the application must be included in the project's URL configuration.

CRUD Formula

```
CREATE -> CreateView
READ   -> ListView / DetailView
UPDATE -> UpdateView
DELETE -> DeleteView
```

---

8. Form Handling with Class-Based Views

Form processing generally has three paths:

1. Initial GET request: Display an empty or prepopulated form.
2. Invalid POST request: Redisplay the form with errors.
3. Valid POST request: Process the data and redirect.

Django provides generic views to simplify form handling.

8.1 FormView

`FormView` handles form display, validation, and successful submission.

It does not automatically save a Django model instance.

forms.py

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    message = forms.CharField(widget=forms.Textarea)

    def send_email(self):
        # Implement email delivery here.
        pass
```

views.py

```
from django.urls import reverse_lazy
from django.views.generic.edit import FormView

from .forms import ContactForm


class ContactView(FormView):
    template_name = "contact.html"
    form_class = ContactForm
    success_url = reverse_lazy("contact-success")

    def form_valid(self, form):
        form.send_email()
        return super().form_valid(form)
```

Important Methods

Method	Purpose
`get_form_class()`	Determines the form class

`get_form()`	Creates a form instance
`get_form_kwargs()`	Supplies arguments to the form constructor
`get_initial()`	Provides initial form data
`form_valid()`	Handles valid submissions
`form_invalid()`	Handles invalid submissions
`get_success_url()`	Determines the redirect destination

The default `form_valid()` implementation redirects to `success_url`.

---

9. Model Forms and Generic Editing Views

Generic editing views can automatically create a `ModelForm`.

Django can determine the model from:

- The `model` attribute.
- The object returned by `get_object()`.
- The model associated with a supplied queryset.

For `CreateView` and `UpdateView`, specify either `fields` or `form_class`.

Do not specify both.

9.1 Using CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView

from .models import Product


class ProductCreateView(CreateView):
    model = Product
    fields = ["name", "price", "description"]
    success_url = reverse_lazy("product-list")
```

9.2 Using a Custom ModelForm

When you need custom validation, create a `ModelForm`.

forms.py

```
from django import forms

from .models import Product


class ProductForm(forms.ModelForm):

    class Meta:
        model = Product
        fields = ["name", "price", "description"]
```

views.py

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
```

```
from .forms import ProductForm
from .models import Product


class ProductCreateView(CreateView):
    model = Product
    form_class = ProductForm
    success_url = reverse_lazy("product-list")
```

When using a custom form class with a generic editing view, specify the model as well when required by the view's configuration.

9.3 `get_absolute_url()`

A model can define `get_absolute_url()` to provide its canonical URL.

```
from django.db import models
from django.urls import reverse


class Author(models.Model):
    name = models.CharField(max_length=200)

    def get_absolute_url(self):
        return reverse(
            "author-detail",
            kwargs={"pk": self.pk},
        )
```

`CreateView` and `UpdateView` can use `get_absolute_url()` when no explicit success URL is supplied and the model provides this method.

9.4 Why Use `reverse_lazy()`?

`reverse()` resolves a named URL immediately.

`reverse_lazy()` defers URL resolution until it is needed.

```
from django.urls import reverse_lazy


class ProductCreateView(CreateView):
    model = Product
```

```
        fields = ["name", "price"]
        success_url = reverse_lazy("product-list")
```

`reverse_lazy()` is particularly useful for class attributes because URL configuration may not be fully loaded when the view module is imported.

---

10. Authentication and Authorization

Authentication and authorization solve different problems.

- **Authentication:** Who is the user?
- **Authorization:** What is the user allowed to do?

Django provides mixins to enforce access restrictions in CBVs.

10.1 LoginRequiredMixin

`LoginRequiredMixin` restricts access to authenticated users.

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView

from .models import Product


class DashboardView(LoginRequiredMixin, ListView):
    model = Product
    template_name = "dashboard.html"
    login_url = "/accounts/login/"
```

Unauthenticated users are normally redirected to the login page.

10.2 PermissionRequiredMixin

`PermissionRequiredMixin` checks whether a user has the required Django permission.

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    PermissionRequiredMixin,
)
from django.urls import reverse_lazy
from django.views.generic import DeleteView

from .models import Product
```

```
class ProductDeleteView(
    LoginRequiredMixin,
    PermissionRequiredMixin,
    DeleteView,
):
    model = Product
    permission_required = "shop.delete_product"
    success_url = reverse_lazy("product-list")
```

Permission strings follow this format:

```
app_label.permission_codename
```

For example:

```
shop.delete_product
```

10.3 UserPassesTestMixin

`UserPassesTestMixin` checks a custom condition before allowing access.

For example, users may update only products they created.

First, assume the `Product` model has a `created_by` field.

```
from django.conf import settings
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )
```

Then implement the view:

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
```

```
from django.urls import reverse_lazy
from django.views.generic import UpdateView

from .models import Product



class ProductUpdateView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    UpdateView,
):
    model = Product
    fields = ["name"]
    success_url = reverse_lazy("product-list")

    def test_func(self):
        product = self.get_object()
        return product.created_by_id == self.request.user.pk
```

Important: `LoginRequiredMixin` should appear before the main generic view in the inheritance list.

For ownership checks, consider filtering the queryset by the current user when appropriate. This can restrict access to other users' objects by returning 404 instead of exposing their existence.

---

11. A Secure Product CRUD Example

Assume `Product` has the following fields:

- `name`
- `price`
- `description`
- `active`
- `created_by`

The `created_by` field is a foreign key to the user model.

11.1 Product Model

```
from django.conf import settings
from django.db import models
```

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    description = models.TextField(blank=True)
    active = models.BooleanField(default=True)

    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )

    def __str__(self):
        return self.name
```

11.2 ProductListView

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView

from .models import Product


class ProductListView(LoginRequiredMixin, ListView):
    model = Product
    template_name = "products/product_list.html"
    context_object_name = "products"

    def get_queryset(self):
        return Product.objects.filter(active=True)
```

This view displays active products to authenticated users.

11.3 ProductDetailView

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import DetailView

from .models import Product
```

```
class ProductDetailView(LoginRequiredMixin, DetailView):
    model = Product
    template_name = "products/product_detail.html"
    context_object_name = "product"
```

This version restricts detail access to authenticated users, but does not restrict users to their own products.

11.4 ProductCreateView

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.urls import reverse_lazy
from django.views.generic import CreateView

from .models import Product



class ProductCreateView(LoginRequiredMixin, CreateView):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

The current user is assigned on the server rather than accepting `created_by` from user input.

11.5 ProductUpdateView

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.urls import reverse_lazy
from django.views.generic import UpdateView

from .models import Product
```

```
class ProductUpdateView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    UpdateView,
):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/product_form.html"
    success_url = reverse_lazy("product-list")

    def test_func(self):
        return self.get_object().created_by_id == self.request.user.pk
```

11.6 ProductDeleteView

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    UserPassesTestMixin,
)
from django.urls import reverse_lazy
from django.views.generic import DeleteView

from .models import Product


class ProductDeleteView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    DeleteView,
):
    model = Product
    template_name = "products/product_confirm_delete.html"
    success_url = reverse_lazy("product-list")

    def test_func(self):
        return self.get_object().created_by_id == self.request.user.pk
```

11.7 Security Checklist

1. Use authentication where required.
2. Check ownership or permissions before modifying records.

3. Explicitly define editable fields.
4. Never trust user-submitted ownership information.
5. Use POST forms with CSRF protection for state-changing actions.
6. Decide whether detail pages should be public or private.
7. Use Django's permission system for role-based access when appropriate.
8. Test unauthorized access to other users' objects.

---

12. Method Overriding in CBVs

Method overriding means providing a custom implementation of a method inherited from a parent class.

It allows you to customize Django's default behavior.

12.1 Overriding `get_queryset()`

`get_queryset()` determines which objects a generic view retrieves.

```
from django.views.generic import ListView

from .models import Product


class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        return Product.objects.filter(active=True)
```

Only active products are returned.

Preserving Parent Behavior

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        queryset = super().get_queryset()
        return queryset.filter(active=True)
```

Use `super()` when you want to retain and extend the inherited behavior.

12.2 Overriding `get_context_data()`

`get_context_data()` adds information to the template context.

```
from django.views.generic import ListView

from .models import Product


class ProductListView(ListView):
    model = Product

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["message"] = "Welcome to the Product List"
        return context
```

Template:

```
<h1>{{ message }}</h1>
```

Best practice: Call `super().get_context_data(**kwargs)` to preserve the context provided by parent classes.

12.3 Overriding `form_valid()`

`form_valid()` runs when a form passes validation.

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.urls import reverse_lazy
from django.views.generic import CreateView

from .models import Product


class ProductCreateView(LoginRequiredMixin, CreateView):
    model = Product
    fields = ["name", "price"]
    success_url = reverse_lazy("product-list")

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

Here, the current user is assigned to the object before Django saves it.

12.4 Overriding `form_invalid()`

`form_invalid()` handles invalid form submissions.

```
    def form_invalid(self, form):
        # Add custom error handling if necessary.
        return super().form_invalid(form)
```

The default behavior normally renders the form again with validation errors.

12.5 Overriding `get_object()`

`get_object()` retrieves a single object.

Often, filtering `get_queryset()` is preferable because it preserves Django's existing object retrieval behavior.

```
    from django.views.generic import DetailView

    from .models import Product


    class ProductDetailView(DetailView):
        model = Product

        def get_queryset(self):
            return Product.objects.filter(active=True)
```

If the requested product is inactive, it will not be available through this view.

---

13. Request Objects and URL Arguments

CBVs expose useful information through the view instance.

13.1 `self.request`

`self.request` contains the current HTTP request.

```
    class ProductListView(LoginRequiredMixin, ListView):
        model = Product

        def get_queryset(self):
            user = self.request.user
            return Product.objects.filter(created_by=user)
```

This retrieves products created by the current user.

13.2 `self.kwargs`

`self.kwargs` contains named URL parameters.

URL configuration:

```
path(
    "products/category/<int:category_id>/",
    ProductListView.as_view(),
    name="products-by-category",
)
```

View:

```
class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        category_id = self.kwargs["category_id"]
        return Product.objects.filter(category_id=category_id)
```

This example assumes that `Product` has a suitable category relationship.

13.3 `self.args`

`self.args` contains positional URL arguments captured by the URL resolver.

Named URL arguments accessed through `self.kwargs` are generally easier to understand and maintain.

---

14. Pagination with ListView

Pagination divides a large list into smaller pages.

```
from django.views.generic import ListView

from .models import Product


class ProductListView(ListView):
    model = Product
    template_name = "products/product_list.html"
```

```
    context_object_name = "products"
    paginate_by = 10
```

The template can use:

- `page_obj` : The current page.
- `paginator` : Pagination metadata.
- `products` : The current page's object list.

Pagination Template

```
<h1>Products</h1>

{% for product in products %}
    <p>{{ product.name }}</p>
{% empty %}
    <p>No products found.</p>
{% endfor %}

<div class="pagination">

    {% if page_obj.has_previous %}
        <a href="?page={{ page_obj.previous_page_number }}">
            Previous
        </a>
    {% endif %}

    <span>
        Page {{ page_obj.number }} of {{ paginator.num_pages }}
    </span>

    {% if page_obj.has_next %}
        <a href="?page={{ page_obj.next_page_number }}">
            Next
        </a>
    {% endif %}

</div>
```

---

15. Dynamic Filtering

You can filter objects using URL parameters, the current user, or other request data.

For example, suppose a book belongs to a publisher.

urls.py

```
from django.urls import path

from .views import PublisherBookListView


urlpatterns = [
    path(
        "books/<str:publisher>/",
        PublisherBookListView.as_view(),
        name="books-by-publisher",
    ),
]
```

views.py

```
from django.shortcuts import get_object_or_404
from django.views.generic import ListView

from .models import Book, Publisher


class PublisherBookListView(ListView):
    model = Book
    template_name = "books/books_by_publisher.html"
    context_object_name = "book_list"

    def get_queryset(self):
        self.publisher = get_object_or_404(
            Publisher,
            name=self.kwargs["publisher"],
        )

        return Book.objects.filter(publisher=self.publisher)

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["publisher"] = self.publisher
        return context
```

The publisher's name comes from the URL.

The view retrieves that publisher and displays the associated books.

---

16. Adding Extra Context

Sometimes a template needs additional information beyond the main object.

For example, a publisher detail page might display its books.

```
from django.views.generic import DetailView

from .models import Book, Publisher


class PublisherDetailView(DetailView):
    model = Publisher
    context_object_name = "publisher"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)

        context["book_list"] = Book.objects.filter(
            publisher=self.object
        )

        return context
```

Here:

- `self.object` is the publisher being displayed.
- `book_list` contains books belonging to that publisher.
- `super()` preserves the context supplied by the parent view.

---

17. Mixins in Django CBVs

17.1 What Is a Mixin?

A **mixin** is a class that provides reusable functionality to another class through inheritance.

Mixins are commonly used for:

- Authentication

- Permission checking
- Form handling
- Context management
- Single-object retrieval
- Multiple-object retrieval
- Template rendering

17.2 Common Mixins

Mixin	Purpose
`LoginRequiredMixin`	Requires authentication
`PermissionRequiredMixin`	Checks Django permissions
`UserPassesTestMixin`	Checks a custom condition
`FormMixin`	Provides reusable form-handling behavior
`ModelFormMixin`	Provides model-form behavior

`SingleObjectMixin`	Handles a single object
`MultipleObjectMixin`	Handles multiple objects
`ContextMixin`	Provides context-related behavior
`TemplateResponseMixin`	Provides template-response behavior

17.3 Combining Mixins

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView


class DashboardView(LoginRequiredMixin, ListView):
    model = Product
```

The mixin provides authentication behavior, while `ListView` provides object-list functionality.

Important Rule

Generally, combine one main generic view with compatible mixins.

Avoid combining multiple generic views that have conflicting methods, such as `get()` or `get_queryset()`, without understanding their method resolution order.

---

18. Understanding `super()` and Method Resolution Order

`super()` allows a class to continue using inherited behavior.

For example:

```
class ProductListView(ListView):
    model = Product

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["message"] = "Product Information"
        return context
```

Without `super()`, you might accidentally discard the context provided by the parent class.

Multiple Inheritance

```
class MyView(LoginRequiredMixin, ListView):
    model = Product
```

Python uses the Method Resolution Order (MRO) to determine which inherited implementation is called.

When working with multiple mixins, the order of inheritance matters.

---

19. Applying Decorators to CBVs

Function-based decorators can be applied to class-based views.

19.1 Applying a Decorator in URL Configuration

```
from django.contrib.auth.decorators import login_required
from django.urls import path
from django.views.generic import TemplateView


urlpatterns = [
    path(
        "secret/",
        login_required(
```

```
                TemplateView.as_view(
                    template_name="secret.html"
                )
            ),
            name="secret",
        ),
    ]
```

This protects the configured view instance.

19.2 Applying a Decorator to the Entire Class

Use `method_decorator` to apply a function decorator to a class method.

```
from django.contrib.auth.decorators import login_required
from django.utils.decorators import method_decorator
from django.views import View


@method_decorator(login_required, name="dispatch")
class ProtectedView(View):

    def get(self, request, *args, **kwargs):
        from django.http import HttpResponse
        return HttpResponse("Authenticated users only.")
```

Decorating `dispatch()` protects all supported HTTP handlers that pass through it.

For ordinary CBV authentication, `LoginRequiredMixin` is often clearer.

19.3 Decorating a Specific Method

```
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse
from django.utils.decorators import method_decorator
from django.views import View


class MyView(View):

    @method_decorator(login_required)
    def get(self, request, *args, **kwargs):
        return HttpResponse("Protected GET request.")
```

This protects only the `get()` method. Other handlers require their own protection if necessary.

---

20. Asynchronous Class-Based Views

Django supports asynchronous CBVs using `async def`.

```
import asyncio

from django.http import HttpResponse
from django.views import View


class AsyncView(View):

    async def get(self, request, *args, **kwargs):
        await asyncio.sleep(1)
        return HttpResponse("Hello async world!")
```

Important Rules

1. Async handlers can use `await` with awaitable operations.
2. All user-defined HTTP handler methods in a single view class must be synchronous or asynchronous; mixing `def` and `async def` handlers causes `ImproperlyConfigured`.
3. Django detects asynchronous handlers and runs the view in an appropriate context.
4. Async views are especially useful for supported asynchronous I/O operations.

Important: Using `async def` alone does not automatically make synchronous database operations or other blocking operations non-blocking. Use Django's supported asynchronous APIs where appropriate.

---

21. Supporting HEAD Requests

A HEAD request asks for the response headers without the response body.

Django's base `View` supports HEAD requests through its default behavior when a `get()` handler exists. You can also define a custom `head()` method.

Example:

```
from django.http import HttpResponse
from django.views import View


class ExampleView(View):

    def get(self, request, *args, **kwargs):
        return HttpResponse("Response body")

    def head(self, request, *args, **kwargs):
        return HttpResponse(
            headers={"X-Status": "OK"}
        )
```

In an API, headers can provide metadata without transferring the full response body.

For example, a `Last-Modified` header can indicate when a resource was last updated.

---

22. Date-Based Generic Views

Django provides date-based views in:

```
django.views.generic.dates
```

These views are useful for blogs, news websites, archives, and event listings.

Assume a `Post` model has a date field called `published_at`.

22.1 ArchiveIndexView

Displays an archive index of date-based objects.

```
from django.views.generic.dates import ArchiveIndexView

from .models import Post


class PostArchiveView(ArchiveIndexView):
    model = Post
    date_field = "published_at"
```

22.2 YearArchiveView

Displays objects from a specified year.

```
from django.views.generic.dates import YearArchiveView

from .models import Post


class PostYearArchiveView(YearArchiveView):
    model = Post
    date_field = "published_at"
    make_object_list = True
```

22.3 MonthArchiveView

Displays objects from a specified month.

```
from django.views.generic.dates import MonthArchiveView

from .models import Post


class PostMonthArchiveView(MonthArchiveView):
    model = Post
    date_field = "published_at"
```

22.4 WeekArchiveView

Displays objects from a specified week.

```
from django.views.generic.dates import WeekArchiveView

from .models import Post


class PostWeekArchiveView(WeekArchiveView):
    model = Post
    date_field = "published_at"
```

22.5 DayArchiveView

Displays objects from a specified day.

```
from django.views.generic.dates import DayArchiveView

```

```
from .models import Post


class PostDayArchiveView(DayArchiveView):
    model = Post
    date_field = "published_at"
```

22.6 TodayArchiveView

Displays objects from the current day.

```
from django.views.generic.dates import TodayArchiveView

from .models import Post


class TodayPostView(TodayArchiveView):
    model = Post
    date_field = "published_at"
```

The URL patterns must supply the appropriate date parameters for the selected archive view.

Templates, date formats, and the model's date field must be configured correctly.

---

23. Authentication Generic Views

Django provides built-in authentication views in:

```
django.contrib.auth.views
```

These views support login, logout, password changes, and password-reset workflows.

23.1 LoginView

```
from django.contrib.auth.views import LoginView


class UserLoginView(LoginView):
    template_name = "accounts/login.html"
```

The template must render the login form and include a CSRF token.

23.2 LogoutView

```
from django.contrib.auth.views import LogoutView


class UserLogoutView(LogoutView):
    next_page = "/"
```

In modern Django versions, logout is performed through POST requests. Use a POST form rather than a state-changing link.

23.3 PasswordChangeView

```
from django.contrib.auth.views import PasswordChangeView
from django.urls import reverse_lazy


class MyPasswordChangeView(PasswordChangeView):
    template_name = "accounts/password_change.html"
    success_url = reverse_lazy("password-change-done")
```

23.4 Other Authentication Views

View	Purpose
`PasswordChangeDoneView`	Password change confirmation
`PasswordResetView`	Starts the password-reset process
`PasswordResetDoneView`	Confirms the reset request
`PasswordResetCo`	Allows the user to set

`nfirmVie w`	a new password
`Passwor dResetCo mpleteVi ew`	Confirms successful password reset

These views require appropriate URL patterns, templates, email configuration, and redirects.

---

24. Returning JSON Responses from CBVs

CBVs can return JSON instead of rendering HTML.

For a simple JSON response, use `JsonResponse`.

```
from django.http import JsonResponse
from django.views import View


class ProductAPIView(View):

    def get(self, request, *args, **kwargs):
        return JsonResponse({
            "message": "Success",
            "status": "ok",
        })
```

`JsonResponse` serializes a Python dictionary into a JSON HTTP response.

For a collection of database objects:

```
from django.http import JsonResponse
from django.views import View

from .models import Product


class ProductAPIView(View):
```

```
        def get(self, request, *args, **kwargs):
            products = list(
                Product.objects.values(
                    "id",
                    "name",
                    "price",
                )
            )

            return JsonResponse({
                "products": products,
            })
```

Using `values()` selects the requested fields and returns dictionaries that can be serialized.

For complex REST APIs, consider Django REST Framework, which provides serializers, API-specific views, authentication, permissions, and other features.

---

25. Content Negotiation

Content negotiation allows a view to return different response formats according to the client's preferences.

For example, a client might request HTML or JSON.

Django provides `HttpRequest.get_preferred_type()` in Django 5.2 and later.

A simplified example:

```
from django.http import HttpResponse, JsonResponse
from django.views import View


class ProductResponseView(View):

    def get(self, request, *args, **kwargs):
        accepted_type = request.get_preferred_type([
            "text/html",
            "application/json",
        ])

        if accepted_type == "application/json":
```

```
                return JsonResponse({
                    "message": "Success",
                })

            if accepted_type == "text/html":
                return HttpResponse(
                    "<h1>Success</h1>"
                )

            return HttpResponse(
                "Not Acceptable",
                status=406,
            )
```

This is a simplified example. Production applications should also consider the response format for unsupported media types and cache behavior.

---

26. Database Query Optimization

Django CBVs do not automatically optimize every database query.

Optimization depends on the relationships accessed and the queries executed.

26.1 `select_related()`

Use `select_related()` for single-valued relationships:

- `ForeignKey`
- `OneToOneField`

Example:

```
from django.views.generic import ListView

from .models import Product



class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        return Product.objects.select_related("category")
```

This assumes that `Product` has a foreign key named `category`.

`select_related()` uses SQL joins to retrieve related objects efficiently.

26.2 `prefetch_related()`

Use `prefetch_related()` for relationships that may return multiple objects:

- `ManyToManyField`
- Reverse foreign-key relationships
- Other supported multi-valued relationships

Example:

```
from django.views.generic import ListView

from .models import Product


class ProductListView(ListView):
    model = Product

    def get_queryset(self):
        return Product.objects.prefetch_related("tags")
```

This assumes that `Product` has a relationship named `tags`.

`prefetch_related()` generally performs additional queries and combines the results in Python.

26.3 Other Optimization Techniques

Technique	Purpose
`select_related()`	Efficiently loads single-valued related objects
`prefetch_relate`	Efficiently loads

`d()`	related collections
`only()`	Initially loads selected fields
`defer()`	Defers loading selected fields
`paginate_by`	Limits objects displayed per page
Caching	Reuses eligible previously computed results

Measure actual query counts and execution times before optimizing.

Django Debug Toolbar can help identify unnecessary queries during development.

---

27. Using SingleObjectMixin with View

Sometimes you need a custom HTTP handler but also need to retrieve an object using Django's generic functionality.

`SingleObjectMixin` provides `get_object()`.

```
from django.http import HttpResponseForbidden, HttpResponseRedirect
from django.urls import reverse
from django.views import View
```

```
from django.views.generic.detail import SingleObjectMixin

from .models import Author


class RecordInterestView(SingleObjectMixin, View):
    model = Author

    def post(self, request, *args, **kwargs):
        if not request.user.is_authenticated:
            return HttpResponseForbidden()

        self.object = self.get_object()

        # Record the user's interest here.

        return HttpResponseRedirect(
            reverse(
                "author-detail",
                kwargs={"pk": self.object.pk},
            )
        )
```

URL configuration:

```
path(
    "author/<int:pk>/interest/",
    RecordInterestView.as_view(),
    name="author-interest",
)
```

This example assumes an `Author` model and an `author-detail` URL pattern.

`SingleObjectMixin` retrieves the object, while `View` provides the HTTP request-handling structure.

---

28. Common Mistakes in Django CBVs

Mistake 1: Forgetting `as_view()`

Incorrect:

```
path("products/", ProductListView)
```

Correct:

```
path(
    "products/",
    ProductListView.as_view(),
    name="product-list",
)
```

Mistake 2: Assuming FormView Automatically Saves a Model

`FormView` handles form processing but does not automatically create or update a model instance.

Use `CreateView`, `UpdateView`, or explicit save logic when necessary.

Mistake 3: Forgetting Editable Fields

`CreateView` and `UpdateView` need a suitable form configuration.

Use either `fields` or `form_class`.

Mistake 4: Replacing the Parent Context Accidentally

Avoid:

```
def get_context_data(self, **kwargs):
    return {"message": "Hello"}
```

This discards the existing context.

Prefer:

```
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["message"] = "Hello"
    return context
```

Mistake 5: Forgetting `super()`

When overriding methods, skipping the parent implementation may remove important behavior.

Call `super()` when inherited behavior should remain part of the execution.

Mistake 6: Confusing Authentication with Authorization

A logged-in user should not automatically be allowed to modify every record.

Check permissions and object ownership when required.

Mistake 7: Performing Deletion Through GET

Use a POST confirmation form with CSRF protection for destructive operations.

Mistake 8: Assuming Query Optimization Is Automatic

Use `select_related()` and `prefetch_related()` where appropriate, and inspect actual queries.

Mistake 9: Combining Incompatible Generic Views

Combining multiple generic views with overlapping methods can produce unexpected behavior.

Understand inheritance, `super()`, and the Method Resolution Order before combining complex mixins.

---

29. Important CBV Methods — Quick Reference

Base View

Method	Purpose
`as_view()`	Creates a callable view
`setup()`	Initializes request attributes
`dispatch()`	Routes the request to a handler
`http_method_not_allowed()`	Handles unsupported methods

`options()`	Handles OPTIONS requests

ListView

Method or Attribute	Purpose
`get_queryset()`	Determines the objects to display
`get_context_data()`	Adds template context
`get_paginate_by()`	Determines pagination size
`paginate_queryset()`	Applies pagination
`context_object_name`	Sets the object-list variable name

DetailView

Method	Purpose
`get_object()`	Retrieves one object
`get_queryset()`	Defines eligible objects
`get_context_data()`	Adds template context

Form and Editing Views

Method	Purpose
`get_form_class()`	Determines the form class
`get_form()`	Creates a form instance
`get_form_kwargs()`	Supplies form constructor arguments
`get_initial()`	Provides initial form data

`form_valid()`	Handles valid form submissions
`form_invalid()`	Handles invalid form submissions
`get_success_url()`	Determines the success redirect

---

30. Interview and Exam Questions

Basic Questions

1. What is a Class-Based View in Django?
2. What is the difference between FBV and CBV?
3. What is the purpose of `as_view()`?
4. What does `dispatch()` do?
5. What is a generic view?
6. Why is `super()` useful in CBVs?

Generic Views

7. What is the difference between `ListView` and `DetailView`?
8. What is the difference between `CreateView` and `UpdateView`?
9. How does `DeleteView` work?
10. When should you use `TemplateView`?
11. What is the purpose of `FormView`?
12. What is `get_queryset()`?
13. What is `get_object()`?

14. How does `get_context_data()` work?
15. What is the purpose of `form_valid()`?

Mixins and Security

16. What is a mixin?
17. What is the difference between `LoginRequiredMixin` and `PermissionRequiredMixin`?
18. When should you use `UserPassesTestMixin`?
19. Why are authentication and authorization different?
20. How can you restrict users to updating their own objects?
21. Why should state-changing operations not use GET requests?

Advanced Questions

22. Explain the CBV request lifecycle.
23. How can you pass extra data to a form?
24. What is the difference between `reverse()` and `reverse_lazy()`?
25. How do `select_related()` and `prefetch_related()` differ?
26. How do you return JSON from a CBV?
27. When should you choose FBV instead of CBV?
28. How can you customize a generic view's queryset?
29. How does pagination work in `ListView`?
30. How can decorators be applied to CBVs?
31. What is the purpose of `get_context_data()`?
32. What is the difference between `FormView` and `CreateView`?
33. How does `UserPassesTestMixin` work?
34. What is the purpose of `LoginRequiredMixin`?
35. How do asynchronous CBVs differ from synchronous CBVs?

---

31. Final Revision Sheet

Core Views

View	Purpose
`View`	Basic request

	handling
`TemplateView`	Render a template
`RedirectView`	Redirect to another URL
`FormView`	Process a form
`ListView`	Display multiple objects
`DetailView`	Display one object
`CreateView`	Create an object
`UpdateView`	Update an object
`DeleteView`	Delete an object

Date-Based Views

- `ArchiveIndexView` — Archive index
- `YearArchiveView` — Yearly archive
- `MonthArchiveView` — Monthly archive
- `WeekArchiveView` — Weekly archive
- `DayArchiveView` — Daily archive
- `TodayArchiveView` — Today's archive

Authentication Views

- `LoginView` — User login
- `LogoutView` — User logout
- `PasswordChangeView` — Change password
- `PasswordChangeDoneView` — Password change confirmation
- `PasswordResetView` — Start password reset
- `PasswordResetDoneView` — Reset request confirmation
- `PasswordResetConfirmView` — Set a new password
- `PasswordResetCompleteView` — Reset completion

Important Mixins

- `LoginRequiredMixin` — Require authentication
- `PermissionRequiredMixin` — Check Django permissions
- `UserPassesTestMixin` — Check a custom condition
- `FormMixin` — Reusable form handling
- `ModelFormMixin` — Model-form behavior
- `SingleObjectMixin` — Single-object handling
- `MultipleObjectMixin` — Multiple-object handling
- `ContextMixin` — Context-related behavior
- `TemplateResponseMixin` — Template-response behavior

Important Methods

- `as_view()` — Creates a callable view
- `setup()` — Initializes the view
- `dispatch()` — Routes HTTP requests
- `get_queryset()` — Determines which objects are retrieved
- `get_object()` — Retrieves a single object
- `get_context_data()` — Adds template context
- `get_form()` — Creates a form instance
- `form_valid()` — Handles valid submissions
- `form_invalid()` — Handles invalid submissions
- `get_success_url()` — Determines the success redirect

CBV Request Flow

```
URL Pattern
     |
     v
```

```
as_view()
     |
     v
View Instance
     |
     v
setup()
     |
     v
dispatch()
     |
     v
HTTP Handler
(get / post / other supported methods)
     |
     v
Generic View Logic
     |
     v
HTTP Response
```

Recommended Learning Order

1. Understand `View`, `as_view()`, `setup()`, and `dispatch()`.
2. Learn `TemplateView`, `RedirectView`, and `FormView`.
3. Master `ListView` and `DetailView`.
4. Practice `CreateView`, `UpdateView`, and `DeleteView`.
5. Learn method overriding and `super()`.
6. Understand authentication, permissions, and object-level authorization.
7. Learn mixins and the Method Resolution Order.
8. Practice pagination and dynamic filtering.
9. Learn database query optimization.
10. Explore asynchronous views and JSON responses.

Final Summary

Django Class-Based Views combine Python classes, inheritance, mixins, and generic views to make web application development more reusable and maintainable.

The most important principle is:

Understand the default behavior of a generic view first, then override only the methods needed to customize it.